Vestige (MCP memory server)

Status: running; exact-handle retrieval only. The installed server is `vestige-mcp 6.0.0` (the AUR package is `vestige-git 5.0.1.r50.g5264387-1`). It stores memory in Strata, an append-only, hash-chained log under `~/.local/share/vestige/log/`. This build has no embedding runtime.

What is installed

Piece	Value
Binary	`/usr/bin/vestige-mcp`
Version	`vestige-mcp 6.0.0`
Transport	stdio, or a Unix-socket daemon shared by clients
Data directory	`~/.local/share/vestige`
Store	Strata: an append-only log of hash-chained frames
Retrieval	exact handle only: full id, unique id prefix of 8+ characters, or exact tag
Embedding runtime	none (`embeddingsCompiledIn: false`)
OpenCode config	`~/.config/opencode/opencode.json` -> `mcp.vestige`

A healthy empty store is a normal state. `memory_status` reports `status: "empty"`, `totalMemories: 0`, and the warning `No embedding runtime in this build - memories are found by exact handle only`.

The `just` coupling

`just mcp-check` opens a real MCP session against Vestige and reads its health (`justfile:218-248`). It calls `initialize`, `memory_status`, and `session_start`, and fails when no answer comes back. An empty store passes.

That recipe is a **liveness and session-context gate, not an ingestion path**. `session_start` assembles status, open intentions, and code context; it writes nothing. The recipe does not save conversation turns, and no other part of this repository does either. Persistence is explicit.

What works

- **Persistence.** `smart_ingest` stores a note or up to twenty items; `codebase remember_pattern` and `codebase remember_decision` store durable code knowledge with anchors. Tags are the index.
- **Retrieval by handle.** `recall` takes a full memory id, a unique prefix of at least eight characters, or an exact tag. It returns the node and its receipt-backed history.
- **Causal retrieval.** `causal_walk` follows recorded edges backward from a failing test, stack frame, CI run, version range, or logged write.
- **Associative retrieval without vectors.** `backfill` raises or lowers earlier memories that share an exact entity with a new memory; `ghostlink` proposes never-composed pairs over recorded typed edges and `weave` records the outcome.
- **Evidence.** every write returns an `eff-` receipt, and `receipt replay` re-derives the state from the log.

A checkout-level smoke test: a note tagged `vestige-check` was written with `smart_ingest`, then retrieved twice, once by the exact tag and once by its full id. `just mcp-check` subsequently reported the same store as `1 memories`.

Semantic retrieval

Semantic similarity search is **not deprecated** it was **removed by design in 4.0** when the engine moved to Strata. The current binary has no embeddings, no cosine index, no BM25, no full-text search, and no keyword matching. A free-text `recall` returns:

```
similarity_disabled: hybrid_search_filtered: embeddings, cosine, BM25, FTS,
Jaccard, and keyword or name matching are not Strata operations; pass an exact
handle
```

`FASTEMBED_CACHE_PATH`, `ORT_DYLIB_PATH`, `HF_HOME`, and `VESTIGE_DISABLE_VECTOR_SEARCH` are no-ops in this build. The `embeddings` and `vector-search` Cargo features are empty stubs that select no crates. There is no flag, environment variable, or `just` recipe that turns semantic retrieval on in 6.0.0.

Upstream's replacement for resemblance search is a retrieval model, not a feature: **exact handles, exact shared entities, and recorded edges**. In practice:

- Resolve an entity or topic to an exact tag, then `recall` that tag.
- Use `backfill` with exact shared entities such as `person:Maya` or `project:Scale`; it does not rank by text similarity.
- Use `causal_walk` for "what led here?" and `ghostlink` for "what has never been combined?".
- Use `codebase get_context` for anchored code knowledge.

If semantic search is a hard requirement, it needs a different engine or an external index in front of Vestige:

1. Keep 6.0.0 authoritative and add a local embedding index (for example `sqlite-vec` or LanceDB). Store `(vector, memory-id, tags)`, search the index, then call `recall(handle=<memory-id>)`.
2. Run legacy v3.1.1 as a separate store. It has the old local Nomic embeddings, but it uses SQLite rather than Strata, cannot share this log, and is not the official replacement path.

What went wrong here

The first check reported an empty store because nothing had ever been ingested: `mcp-check` starts a session and reads health, and conversation turns are not captured automatically. Free-text retrieval then failed because the build has no embedding runtime. Exact retrieval was never broken; it worked as soon as a memory existed.

The operating rule is therefore: **save deliberately, tag for the index, and recall by handle**. Do not expect Vestige to search conversation history by meaning, and do not add an embedding step to `just`; that cannot work in this build.

References

- Upstream `docs/CONFIGURATION.md` — offline-by-default, the no-op embedding variables, and the empty `embeddings`/`vector-search` features.
- Upstream `docs/REFERENCE.md` — "Recall by handle, not resemblance", Retroactive Salience Backfill, GhostLink, and the 4.x architecture.
- `justfile` — the `mcp-check` and `mcp-verify` recipes.
- `AGENTS.md` — the agent-facing rule for saving and recalling memory.